IPL ANALYTICS HUB

Exploratory Data Analysis Dashboard

Technical Documentation & Code Walkthrough

A professional-grade interactive visualization platform analyzing
over 1,000 Indian Premier League matches and ball-by-ball statistics (2008 - 2024).
Built with Flask Python backend, Pandas data pipeline, Seaborn visual layer,
and a cyber-sports themed dark glassmorphism HTML+JS frontend.

Course: Exploratory Data Analysis

Instructor: Ali Hassan Sherazi

Submission Date: June 05, 2026

Developed for academic evaluation and exploratory data research.

Table of Contents

1. Project Architecture & Overview

The IPL Exploratory Data Analysis Dashboard is designed as a decoupled, single-page application comprising a Python Flask web service and an interactive, client-side HTML5/CSS3/JavaScript frontend. By avoiding heavier visualization frameworks like Streamlit or Dash, this architecture yields extremely fast load times, modular separation of concerns, and full control over styling.

The system parses two main datasets: matches.csv, which contains 1,000+ match metadata records from 2008 to 2024, and deliveries.csv, a large-scale database of ball-by-ball details. The backend loads these datasets using Pandas, cleanses discrepancies (such as team rebrandings and missing city names), and aggregates statistics based on active filters. It then generates visual charts using Seaborn and Matplotlib, renders them to base64 images in memory, and transmits them to the frontend as a JSON payload. This ensures that the web dashboard does not require writing files to disk, enabling seamless deployment on serverless environments like Vercel.

2. Backend Architecture Deep Dive

app.py: Flask Server and Endpoint Routing

The main application runner file is app.py. It initializes the Flask framework and exposes endpoints for both serving static assets (HTML, CSS, JS) and running the JSON API. Using Flask's send_from_directory functionality, the server acts as a unified provider, which facilitates simple local execution. The file defines two critical API endpoints:
1. /api/filters (GET): Collects the overall dataset boundaries (such as season lists, team names, city names, and maximum margins) from the data layer and serves them to the frontend to build the filters dynamically.
2. /api/dashboard (POST): Receives the user's active filter values as a JSON payload, redirects them to the filters module to obtain a filtered DataFrame, computes KPI metrics, generates base64 image strings from charts.py, and returns a unified JSON response.

Code Snippet: Flask Server Definition (app.py)

```
import os
from flask import Flask, jsonify, request, send_from_directory
from filters import get_filter_options, filter_data, calculate_kpis
import charts

app = Flask(__name__)

@app.route('/')
def index():
    return send_from_directory('.', 'index.html')

@app.route('/api/filters', methods=['GET'])
def api_filters():
    return jsonify(get_filter_options())

@app.route('/api/dashboard', methods=['POST'])
def api_dashboard():
    filters = request.json or {}
    filtered_matches, filtered_deliveries = filter_data(filters)
    kpis = calculate_kpis(filtered_matches, filtered_deliveries)

    chart_images = {
        "toss_decision": charts.get_toss_decision_chart(filtered_matches),
        "target_runs_hist": charts.get_target_runs_histogram(filtered_matches),
        "matches_trend": charts.get_matches_trend_chart(filtered_matches),
        "top_winners": charts.get_top_winners_chart(filtered_matches),
        "target_vs_margin": charts.get_target_vs_margin_scatter(filtered_matches),
        "margin_box": charts.get_margin_boxplot(filtered_matches),
        "correlation_heat": charts.get_correlation_heatmap(filtered_matches),
        "cumulative_wins": charts.get_cumulative_wins_area(filtered_matches),
        "cities_count": charts.get_cities_countplot(filtered_matches),
        "target_violin": charts.get_target_runs_violin(filtered_matches)
    }
    return jsonify({"kpis": kpis, "charts": chart_images, "record_count": len(filtere
      d_matches)})
```

filters.py: Data Preprocessing and Dynamic Query Filtering

Data cleaning and dynamic querying are handled exclusively by filters.py. The module standardizes team names to account for franchises rebranding over the years (e.g., Delhi Daredevils renamed to Delhi Capitals, or Kings XI Punjab to Punjab Kings). It also resolves missing city data by checking a dictionary mapping venues to cities. Furthermore, to save system memory and speed up processing, deliveries.csv is read selectively, importing only the columns required for dashboard aggregations.

Code Snippet: Data Cleaning and Loading (filters.py)

```
TEAM_STANDARDIZATION = {
    "Delhi Daredevils": "Delhi Capitals",
    "Kings XI Punjab": "Punjab Kings",
    "Rising Pune Supergiants": "Rising Pune Supergiant",
    "Royal Challengers Bengaluru": "Royal Challengers Bangalore"
}

SEASON_MAPPING = {"2007/08": "2008", "2009/10": "2010", "2020/21": "2020"}

def load_and_clean_data():
    global _matches_df, _deliveries_df
    if _matches_df is not None and _deliveries_df is not None:
        return _matches_df, _deliveries_df

    matches = pd.read_csv(MATCHES_PATH)
    matches['season'] = matches['season'].replace(SEASON_MAPPING)
    matches['date'] = pd.to_datetime(matches['date'], errors='coerce')

    # Impute cities
    matches['city'] = matches.apply(lambda r: venue_city_map.get(r['venue'], 'Other')
       if pd.isna(r['city']) else r['city'], axis=1)

    # Standardize teams
    for col in ['team1', 'team2', 'toss_winner', 'winner']:
        matches[col] = matches[col].replace(TEAM_STANDARDIZATION)
    matches['winner'] = matches['winner'].fillna('No Result')
    matches['result_margin'] = matches['result_margin'].fillna(0)

    deliveries = pd.read_csv(DELIVERIES_PATH, usecols=['match_id', 'inning', 'batting
      _team', 'bowling_team', 'total_runs', 'is_wicket', 'batsman_runs'])
    deliveries['batting_team'] = deliveries['batting_team'].replace(TEAM_STANDARDIZAT
      ION)
    deliveries['bowling_team'] = deliveries['bowling_team'].replace(TEAM_STANDARDIZAT
      ION)

    _matches_df, _deliveries_df = matches, deliveries
    return _matches_df, _deliveries_df
```

charts.py: Visualization Design and Base64 Encoding

The charts.py file contains the code responsible for generating the visual representation of data. It configures a dark sports theme matching the frontend using Matplotlib's rcParams and Seaborn's theme settings. To run reliably on servers where no graphic display window is available, it enforces the non-interactive Agg backend (`matplotlib.use('Agg')`).

The key utility in charts.py is fig_to_base64. It saves the plotted Matplotlib figure into an in-memory BytesIO buffer as a PNG, encodes it as a base64 string, closes the plot to free RAM, and returns the string. This lets the Flask backend send the raw image bytes in standard JSON text, avoiding file IO entirely. If a filter results in an empty DataFrame, draw_empty_chart is invoked, displaying a user-friendly 'No Data Matching Filters' warning instead of throwing an error.

Code Snippet: Chart Styling & Base64 Encoder (charts.py)

```
import io
import base64
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import seaborn as sns

THEME_BG = '#16171d'
CARD_BG = '#20212a'
TEXT_COLOR = '#ffffff'

def apply_chart_style():
    plt.rcParams['figure.facecolor'] = THEME_BG
    plt.rcParams['axes.facecolor'] = CARD_BG
    plt.rcParams['text.color'] = TEXT_COLOR
    plt.rcParams['axes.labelcolor'] = TEXT_COLOR
    plt.rcParams['xtick.color'] = TEXT_COLOR
    plt.rcParams['ytick.color'] = TEXT_COLOR
    sns.set_theme(style="dark", rc={
        "figure.facecolor": THEME_BG,
        "axes.facecolor": CARD_BG,
        "text.color": TEXT_COLOR,
        "axes.labelcolor": TEXT_COLOR,
        "xtick.color": TEXT_COLOR,
        "ytick.color": TEXT_COLOR
    })

def fig_to_base64(fig):
    buf = io.BytesIO()
    fig.savefig(buf, format='png', bbox_inches='tight', dpi=110, facecolor=THEME_BG)
    buf.seek(0)
    img_str = base64.b64encode(buf.read()).decode('utf-8')
    plt.close(fig)
    return img_str
```

3. Frontend Design Deep Dive

index.html: Structuring the Responsive Dashboard

The dashboard layout is defined in index.html. It uses a modern two-column layout consisting of a collapsible sidebar on the left containing all filters (text search, season selects, date picks, victory margin sliders, and checkboxes for teams) and a scrollable main content area on the right. The main panel contains a dynamic match counter badge, a row of five KPI summary cards designed to show core metrics (Total Matches, Total Runs Scored, Average Target runs, Toss Impact Win %, and Top Player), and a responsive grid layout displaying the 10 chart containers. Each chart card features an info-tooltip describing the plot and a full-screen loading spinner overlay.

style.css: The Glassmorphism Cyber-Sports UI

To stand out from basic administrative dashboards, the style.css file defines a rich, sports-themed dark interface. It leverages CSS custom properties (variables) to maintain color scheme consistency across components (Accents in Cyan #00f2fe, Purple #8a2be2, Gold #ffd700, and Coral #ff7f50). The styling highlights consist of semi-transparent card panels (`rgba(32, 33, 42, 0.65)`), modern borders, backdrop filters for a frosted-glass blur effect (`backdrop-filter: blur(12px)`), glow animations on hover, and responsive grid structures (`grid-template-columns: repeat(auto-fit, minmax(450px, 1fr))`) to adapt gracefully to mobile, tablet, and desktop views.

script.js: AJAX Pipeline and DOM Reconciliation

The script.js handles all client-side logic. On startup, it triggers a fetch GET call to `/api/filters` to retrieve dynamic constraints (list of seasons, teams, players) and populates the select lists and checkbox arrays. It binds event listeners to all interactive inputs. When an input changes, it collects all active state variables, packs them into a single JSON payload, and posts to `/api/dashboard`. To avoid multiple concurrent network queries while typing in the keyword search box, the script implements a 450ms debounce mechanism. Once a response is returned, script.js updates the text nodes for the KPIs and sets the `src` attribute of the image placeholders to the newly received base64 string bytes, which triggers a smooth fading transition.

4. Comprehensive Chart Specifications and Data Mapping

Chart 1: Toss Decision Distribution (Pie Chart)

- **Purpose:** To determine the strategic preferences of team captains upon winning the toss (Batting vs Fielding).
- **Data Source File:** matches.csv
- **Dataset Columns Used:** toss_decision (categorical variable containing values 'bat' or 'field').
- **Code Implementation:** charts.py -> get_toss_decision_chart()
- **Visual Design:** Donut chart utilizing purple (#8a2be2) and cyan (#00f2fe) wedges with a central blanking circle.

Insight: Historically, teams in the IPL prefer fielding first (about 60% of cases), which allows them to exploit the chase factor under dew conditions.

Chart 2: Target Runs Frequency Distribution (Histogram)

- **Purpose:** To analyze the density and score thresholds of first-innings targets.
- **Data Source File:** matches.csv
- **Dataset Columns Used:** target_runs (numerical score set by the team batting first).
- **Code Implementation:** charts.py -> get_target_runs_histogram()
- **Visual Design:** Gold (#ffd700) filled frequency bins with a Seaborn Kernel Density Estimation (KDE) line overlay.

Insight: Scores show a normal-like distribution peaking heavily around 160-180 runs. Scores over 200 or below 120 occur infrequently and represent extreme batting conditions or performance gaps.

Chart 3: Matches Played Over Seasons (Line Chart)

- **Purpose:** To show the chronological expansion and volume of the tournament over the seasons.
- **Data Source File:** matches.csv
- **Dataset Columns Used:** season (standardized 4-digit string represent of season year).
- **Code Implementation:** charts.py -> get_matches_trend_chart()
- **Visual Design:** Cyan line plot with circular gold markers highlighting data coordinates.

Insight: Match spikes indicate league expansions, such as in 2011-2013 and from 2022 onwards, when the league introduced new team franchises.

Chart 4: Top 10 Teams by Victory Count (Bar Chart)

- **Purpose:** To compare team success and identify dominant franchises in the history of the league.
- **Data Source File:** matches.csv
- **Dataset Columns Used:** winner (categorical variable of winning teams, ignoring 'No Result').
- **Code Implementation:** charts.py -> get_top_winners_chart()
- **Visual Design:** Horizontal bar chart colored with Seaborn's viridis color spectrum. Win counts are labelled on the bar ends.

Insight: Highly successful franchises like Mumbai Indians and Chennai Super Kings lead the victory counts, demonstrating sustained competitive excellence.

Chart 5: Target Runs vs. Victory Margin (Scatter Plot)

- **Purpose:** To evaluate the correlation between first-innings target scores and final victory margins.
- **Data Source File:** matches.csv
- **Dataset Columns Used:** target_runs (numerical score), result_margin (numerical margin), result (mode of victory: runs/wickets).
- **Code Implementation:** charts.py -> get_target_vs_margin_scatter()
- **Visual Design:** Two-dimensional scatter plot, colored with coral points for run-victories and cyan points for wicket-victories.

Insight: High-target matches (above 200 runs) that are won by the team batting first ('runs') show a wide margin of victory. Conversely, wicket-based wins are structurally bounded between 1 and 10 wickets, indicating a distinct data distribution.

Chart 6: Victory Margin Spread by Win Type (Box Plot)

- **Purpose:** To compare the variance, quartiles, median, and extreme outliers of victory margins for both win types.
- **Data Source File:** matches.csv
- **Dataset Columns Used:** result (victory mode: runs or wickets), result_margin (winning margin).
- **Code Implementation:** charts.py -> get_margin_boxplot()
- **Visual Design:** Box-and-whisker plot highlighting median line, IQR boxes, and 'x' markers indicating outlier data points.

Insight: Victories by runs have high variance with margins stretching up to 140+ runs. Wicket victory margins are strictly bounded between 1 and 10, showing narrow distributions.

Chart 7: Correlation Heatmap of Match Attributes (Heatmap)

- **Purpose:** To find linear correlations and dependencies between key numerical variables.
- **Data Source File:** matches.csv
- **Dataset Columns Used:** target_runs, result_margin, target_overs, and season (mapped to integer).
- **Code Implementation:** charts.py -> get_correlation_heatmap()
- **Visual Design:** Correlation grid annotated with Pearson coefficient values, using the mako colormap.

Insight: Target runs show a positive correlation with victory margins for runs-won matches. A positive correlation is also seen with season years, indicating that average target scores are rising over time.

Chart 8: Cumulative Wins Trend for Top 4 Teams (Area Chart)

- **Purpose:** To track the rate of win accumulation for the league's top 4 franchises over time.
- **Data Source File:** matches.csv
- **Dataset Columns Used:** season (time scale), winner (identifying wins for the top 4 teams).
- **Code Implementation:** charts.py -> get_cumulative_wins_area()
- **Visual Design:** Stacked area chart filled with cyan (#00f2fe), purple (#8a2be2), gold (#ffd700), and coral (#ff7f50).

Insight: Slopes show team consistency. Steeper slopes represent seasons of high win frequency, while flat sections show slumps.

Chart 9: Top 10 Host Cities by Match Count (Count Plot)

- **Purpose:** To identify major geographic hubs and stadiums hosting the highest volume of IPL matches.
- **Data Source File:** matches.csv
- **Dataset Columns Used:** city (cleaned and imputed geographical string).
- **Code Implementation:** charts.py -> get_cities_countplot()
- **Visual Design:** Horizontal bar plot styled with the crest palette, with match counts displayed beside each bar.

Insight: Major metropolitan centers like Mumbai, Bangalore, and Kolkata dominate the match counts, as they serve as primary venues.

Chart 10: Target Runs Density Spread over Seasons (Violin Plot)

- **Purpose:** To study how first-innings scoring distributions and density profiles have evolved year-over-year.
- **Data Source File:** matches.csv
- **Dataset Columns Used:** season (time variable), target_runs (numerical distribution).
- **Code Implementation:** charts.py -> get_target_runs_violin()
- **Visual Design:** Seaborn violin plot sorted chronologically using the plasma colormap.

Insight: The violins grow taller and wider in recent seasons, showcasing a clear upward shift in scoring averages and higher variability in target scores.

5. Dynamic Filtering Logic & KPI Calculations

To comply with the assignment guidelines, the dashboard connects all interactive filters to all 10 charts. When any filter is changed on the client-side UI, it sends the full state vector to the Flask backend, which executes the filter chain sequentially. Below is the technical description of the filters and calculations:

- **Search Keyword:** Filters matches by substring matching. Performs a case-insensitive search across venue, winner, and player_of_match columns.
- **Season Dropdown:** Filters matches by specific season years. Uses list-inclusion checks to filter rows.
- **Player Select:** Isolates matches where a specific player was awarded the Player of the Match title.
- **Date Range:** Parses start and end dates using pd.to_datetime and filters the matches DataFrame between those boundaries.
- **Min Result Margin:** Filters out matches where the result_margin is less than the slider value.
- **Multi-Select Teams:** Filters matches where the checked teams were involved as either team1 or team2.

KPI Calculations (filters.py -> calculate_kpis())

1. Total Matches: Calculated using the length of the filtered matches DataFrame (len(filtered_matches)).
2. Total Runs Scored: Calculated by selecting the ball-by-ball records in deliveries.csv that correspond to the filtered match IDs, and summing the total_runs column (filtered_deliveries['total_runs'].sum()).
3. Average Target Runs: Calculated by taking the mean of the target_runs column across filtered matches (filtered_matches['target_runs'].mean()).
4. Toss Impact (Win %): Calculated by taking the subset of filtered matches where the toss winner matches the match winner (filtered_matches['toss_winner'] == filtered_matches['winner']), dividing by total matches, and multiplying by 100.
5. Top Player: Determined by counting value frequencies in the player_of_match column, identifying the mode, and retrieving its frequency count (e.g., 'AB de Villiers (23x)').

Code Snippet: KPI Calculations (filters.py)

```
def calculate_kpis(filtered_matches, filtered_deliveries):
    total_matches = len(filtered_matches)
    if total_matches == 0:
        return {"total_matches": 0, "total_runs": 0, "avg_target": 0.0, "highest_marg
      in": 0, "toss_impact": 0.0, "top_player": "N/A"}

    total_runs = int(filtered_deliveries['total_runs'].sum()) if len(filtered_deliver
      ies) > 0 else 0
    avg_target = float(round(filtered_matches['target_runs'].mean(), 1)) if 'target_r
      uns' in filtered_matches.columns else 0.0
    highest_margin = int(filtered_matches['result_margin'].max())

    # Toss impact: % of matches won by the toss winner
    toss_winners = filtered_matches[filtered_matches['toss_winner'] == filtered_match
      es['winner']]
    toss_impact = float(round((len(toss_winners) / total_matches) * 100, 1))

    # Top Player of Match
    top_player_str = "N/A"
```

```
    if 'player_of_match' in filtered_matches.columns:
        potm_counts = filtered_matches['player_of_match'].dropna().value_counts()
        if len(potm_counts) > 0:
            top_player_str = f"{potm_counts.index[0]} ({int(potm_counts.iloc[0])}x)"

    return {
        "total_matches": total_matches,
        "total_runs": total_runs,
        "avg_target": avg_target,
        "highest_margin": highest_margin,
        "toss_impact": toss_impact,
        "top_player": top_player_str
    }
```